

C++ Assignment 2

Topics: Arrays of Objects · Function Overloading · Default Arguments · Default Constructor · Parameterised Constructor · Constructor Overloading · Constructor Initializer List · `const` Members · Destructor · Object Lifecycle · `new` / `delete` / `delete[]` · Dynamic 1D & 2D Arrays · Pointer to Objects · `this` Pointer · Method Chaining · Namespaces · Scope Resolution (`::`) · Separate Compilation (`.h` / `.cpp`)

Question 1 — Inventory Management System

You are hired as a junior C++ developer at a retail company. The warehouse team manages hundreds of product items manually on paper. Your manager asks you to write a small inventory module.

"We need to store product records, look them up, calculate discounts, and print summaries. Also, our existing codebase has separate `add` functions for `int` and `double` quantities — clean that up using function overloading."

Part A — Product class with Array of Objects

Design a class `Product` with the following private data members:

- `int productId`
- `string name`
- `double price`
- `int quantity`

Public member functions:

```
void acceptDetails();           // Read all fields from user
void displayDetails() const;    // Print formatted product info
double totalValue() const;      // price * quantity
bool isLowStock(int threshold) const; // true if quantity < threshold
```

In `main()`:

1. Create an **array of 5 `Product` objects on the stack**
2. Accept details for all 5 using a loop
3. Display all products in a formatted table
4. Print the product with the highest total value
5. Print all products where quantity is below a threshold entered by the user

Expected Output (sample):

```
===== INVENTORY REPORT =====
ID   Name       Price   Qty   Total Value
1001 Widget-A    250.00  120   30000.00
1002 Bolt-B     15.50   8     124.00   ← LOW STOCK
...
```

```
Highest Value Product : Widget-A (Rs. 30,000.00)
Low Stock (threshold: 10) : Bolt-B, ...
```

Part B — Function Overloading

The warehouse system needs to calculate restocking costs for different quantity types. Write the following overloaded functions (not inside any class):

```
double reorderCost(int qty, double unitPrice);           // integer quantity
double reorderCost(double qty, double unitPrice);       // fractional (by weight)
double reorderCost(int qty, double unitPrice, double taxRate); // with tax
```

Also write a function with a **default argument**:

```
double applyDiscount(double price, double discountPercent = 10.0);
// If no discount is passed, apply 10% by default
```

Call all versions from `main()` and print the results.

Question 2 — Hospital Patient Registry

A government hospital is replacing its paper-based patient records system. You are on the team building the Patient module. The system needs to handle patients who walk in with full details, patients registered in emergencies (name and ID only), and bulk ward admission.

"We need proper constructor overloading so the system handles all these cases without separate functions. And since patient records are created and destroyed throughout the day, we need to see the object lifecycle clearly."

Design a class `Patient` with:

Private data members:

- `int patientId`
- `string name`
- `int age`
- `string ward`
- `const string bloodGroup` ← must be set at construction time only

Public member functions:

```
// Constructor 1 – Default: id=0, name="Unknown", age=0, ward="General",
bloodGroup="O+"
Patient();
```

```
// Constructor 2 – Emergency admission: only id and name known
Patient(int id, const string& name);

// Constructor 3 – Full admission details
Patient(int id, const string& name, int age, const string& ward, const string&
bg);

// Destructor – print "Patient <name> discharged." when object is destroyed
~Patient();

void displayRecord() const;
void transferWard(const string& newWard);
```

Use Constructor Initializer List to initialise `bloodGroup` in all three constructors (it is `const` — it cannot be assigned inside the body).

In `main()`:

1. Create one patient of each type (3 stack objects) — observe constructor messages
2. Create a **dynamic array** of 4 patients on the heap using `new Patient[4]` (default constructor)
3. Use a loop to call `displayRecord()` on all 4
4. Transfer one patient's ward using `transferWard()`
5. `delete[]` the array and observe destructor messages
6. Let the 3 stack objects go out of scope and observe destructor messages

Expected Output (sample):

```
[Constructor] Full admission: Meera Joshi
[Constructor] Emergency: Raj Patel
[Constructor] Default patient registered.
[Constructor] Default patient registered.    (x4 for array)

Patient Record:
  ID      : 1001
  Name    : Meera Joshi
  Age     : 34
  Ward    : Cardiology
  Blood Grp : B+

Ward Transfer: Raj Patel → ICU

[Destructor] Patient Meera Joshi discharged.    (x4 array cleanup on delete[])
[Destructor] Patient Raj Patel discharged.      (stack objects, reverse order)
...
```

Question 3 — Game Engine Entity System

You are interning at a game studio. The engine team asks you to implement a basic `Entity` class for game objects (players, enemies, items). The lead developer sends you this note:

"Use `this` pointer in setters so we can chain calls —

`entity.setName("Archer").setHealth(100).setLevel(5)` — this is standard in our codebase. Also, we're moving some math utilities into their own namespace so they don't collide with standard library names. And the matrix (2D grid) for the game map needs dynamic allocation — the size isn't known at compile time."

Part A — `this` Pointer and Method Chaining

Design a class `Entity` with private data members:

- `string name`
- `int health`
- `int level`
- `string type` ("Player", "Enemy", "Item")

All setters must return `Entity&` so calls can be chained:

```
Entity& setName(const string& name);
Entity& setHealth(int health);
Entity& setLevel(int level);
Entity& setType(const string& type);
```

Getters (all `const`):

```
string getName() const;
int getHealth() const;
int getLevel() const;
string getType() const;

void displayInfo() const;
```

In `main()`, create at least 3 entities using method chaining:

```
Entity player, enemy, item;
player.setName("Aragorn").setHealth(100).setLevel(10).setType("Player");
enemy.setName("Orc").setHealth(60).setLevel(5).setType("Enemy");
item.setName("HealthPotion").setHealth(0).setLevel(1).setType("Item");
```

Part B — Namespaces

The engine has two modules: `Physics` and `GameMath`. Both need a `clamp()` function, but for different purposes.

Define both namespaces with their own `clamp()` and `lerp()` functions:

```

namespace Physics {
    double clamp(double val, double min, double max); // clamp a velocity
    double lerp(double a, double b, double t);        // linear interpolation for
physics
}

namespace GameMath {
    int clamp(int val, int min, int max);              // clamp an integer stat
(health, level)
    double lerp(double a, double b, double t);        // lerp for UI animations
}

```

Call both versions from `main()` using `Physics::clamp()` and `GameMath::clamp()`. Also demonstrate `using namespace` in a limited block scope (not at the top of the file).

Part C — Dynamic 2D Game Map

The game map is an `R × C` grid of integers (each cell stores a tile type). `R` and `C` are read from the user at runtime — so the map must be allocated on the heap using `int**`.

Requirements:

1. Allocate the 2D array using `new int*[R]` and then `new int[C]` for each row
2. Fill the grid: accept values from user or use `rand() % 5` to simulate tile types (0=grass, 1=water, 2=mountain, 3=forest, 4=dungeon)
3. Display the grid with a legend
4. Find and print the count of each tile type
5. Deallocate properly: `delete[]` each row, then `delete[]` the outer array

Expected Output (sample, 3×4 random map):

```

===== GAME MAP (3 x 4) =====
 2  0  3  1
 0  4  0  2
 3  1  0  0

Legend: 0=Grass  1=Water  2=Mountain  3=Forest  4=Dungeon

Tile Count:
Grass      : 5
Water      : 2
Mountain   : 2
Forest     : 2
Dungeon    : 1

```

Scope Resolution Bonus (Optional)

The game engine has a global `int level = 1` (game difficulty level) and a local `int level` inside `main()` (player level). Print both values inside `main()` — use the `::` scope resolution operator to access the global one.

Also define a nested namespace `Engine::Audio` with a function `playSound(string name)` that prints `"Playing: <name>"`. Call it using the full path `Engine::Audio::playSound("sword_clash")`.